Vidya Pratishthan's Kamalnayan Bajaj Institute of Engineering and Technology,
Baramati Department of Artificial Intelligence & Data Science
Academic Year: 2024-25

Class: S.E **Subject Name**: Database Management System (AI23212)

Sr. No.	Title of Assignment	
1	Draw ER diagrams for different schemas & Convert them into tables (Assume any suitable schema). Display constraints.	Group A
2	SQL Queries: Design and Develop SQL DDL statements which demonstrate the use of SQL objects such as Table, View, Index, Sequence, Synonym, different constraints etc.	
3	SQL Queries: Write at least 10 SQL queries on the suitable database application using SQL DML statements.	
4	SQL Queries – All types of Join, Sub-Query and View: Write at least10 SQL queries for suitable database application using SQL DML statements.	
5	Unnamed PL/SQLcode block: Use of Control structure & Exception handling is mandatory.	Group B
6	Write a PL/SQL block of code using parameterized Cursor that will merge the data available in the newly created table N_Roll_Call with the data available in the table O_Roll_Call. If the data in the first table already exists in the second table then that data should be skipped.	
7	Design and Develop MongoDB Queries using CRUD operations. (Use CRUD operations, SAVE method, logical operators etc.).	
8	MongoDB – Aggregation and Indexing: Design and Develop MongoDB Queries using aggregation and indexing with suitable example using MongoDB.	
9	Exporting and Importing data: Design and develop SQL DML statements to demonstrate exporting/Importing tables to external files of different file formats ex. CSV, XLSX, TXT, etc.	
10	Database Connectivity: Write a program to implement MySQL/Oracle database connectivity with any front end language to implement Database navigation operations (add, delete, edit etc.)	

EXPERIMENT NO : 01

Aim : Draw ER-Diagrams for different schemas and convert them into tables(Assume any suitable schema). Apply normalization. Display constraints.

Outcome :

1. Clear Database structure
2. Improves communication
3. Efficient Database design
4. Identification of relationships
5. Normalization & Data Integrity

Software Requirements : Flowdia Diagrams, Smart Draw

Theory :

An **ER (Entity-Relationship) Diagram** is a visual representation of the data and relationships within a system. It is widely used in database design to illustrate how entities (data objects) interact with one another. Here's a breakdown of its theory and key components:

- Core Concepts in ER Diagrams

1. **Entities**:
    - Represent real-world objects or concepts that have data stored about them in a database.
    - Types:
        - **Strong Entities**: Exist independently and have a unique identifier (primary key).
        - **Weak Entities**: Depend on another entity for existence and typically have a composite key (a foreign key combined with a partial key).
2. **Attributes**:
    - Represent properties or characteristics of an entity.
    - Types:
        - **Simple Attributes**: Indivisible (e.g., Name, Age).
        - **Composite Attributes**: Can be broken down into smaller parts (e.g., Full Name can be split into First Name and Last Name).
        - **Derived Attributes**: Calculated from other attributes (e.g., Age derived from Date of Birth).
        - **Multi-Valued Attributes**: Can have multiple values (e.g., Phone Numbers).
3. **Relationships**:
    - Describe how two or more entities are associated with one another.
    - Types:
        - **One-to-One (1:1)**: Each instance of Entity A is related to one instance of Entity B.

- **One-to-Many (1:N)**: Each instance of Entity A can relate to multiple instances of Entity B.
- **Many-to-Many (M:N)**: Each instance of Entity A can relate to multiple instances of Entity B, and vice versa.

4. **Keys**:
   - **Primary Key**: Uniquely identifies an entity instance.
   - **Foreign Key**: Used to establish a link between two entities.

➢ Symbols in ER Diagrams

Symbol	Description	Example
Rectangle	Represents an **Entity** (e.g., Person, Product)	`Student`, `Course`
Oval	Represents an **Attribute** of an entity	`Name`, `Age`
Double Oval	Represents a **Multi-Valued Attribute**	`Phone Numbers`
Dashed Oval	Represents a **Derived Attribute**	`Age` (calculated from DOB)
♦ Diamond	Represents a **Relationship** between entities	`Enrolled`, `Works At`
Double Diamond	Represents a **Weak Relationship**	`Dependent of`
Line	Connects entities to relationships or attributes	-
Primary Key (PK)	Underlined in an attribute to indicate a **Primary Key**	`StudentID`
Foreign Key (FK)	Used to represent a key that relates to another entity	`CourseID` in `Enrollment`
Composite Attribute	Attributes grouped inside a single oval	`{Full Name: First, Last}`

➢ Steps to Create an ER Diagram

1. Identify all entities and their attributes.
2. Determine the relationships between entities.
3. Assign keys (primary and foreign) to the entities.
4. Draw the diagram using appropriate symbols and notation.

➢ Importance of ER Diagrams

- Provides a clear overview of the data structure.

- Facilitates communication between developers, analysts, and stakeholders.
- Serves as a blueprint for database creation and optimization.

➢ Example ER Diagram with Symbols

Below is a visual explanation of an ER diagram with symbols:

1. **Entities**:
    - Student (Rectangle)
    - Course (Rectangle)
2. **Attributes**:
    - Student has attributes: StudentID (PK), Name, Age
    - Course has attributes: CourseID (PK), Title
3. **Relationship**:
    - Enrolled (Diamond) connects Student and Course

Conclusion :

The Entity-Relationship (ER) diagram serves as a powerful tool for conceptualizing and designing databases. It provides a clear and structured visualization of the data model, emphasizing the entities, their attributes, and relationships.

By using ER diagrams, developers and stakeholders can collaboratively build robust databases that align with organizational needs, ensuring accuracy, scalability, and maintainability in the data system.

EXPERIMENT NO : 02

Aim : SQL Queries: Design and Develop SQL DDL statements which demonstrate the use of SQL objects such as Table, View, Index, Sequence, Synonym, different constraints etc.

Outcomes :

1. Clear Database structure
2. Improves communication
3. Apply Constraints for Data Integrity
4. Design and Create Tables
5. Implement Views for Data Abstraction

Software Requirements : MySQL Workbench

Theory :

DDL changes the structure of the table like creating a table, deleting a table, altering a table, etc. All the command of DDL are auto-committed that means it permanently save all the changes in the database.

DDL Commands:

1. **Create:**
   This command is used to create a new table in SQL. The user has to give information like table name, column names, and their datatypes.

   Syntax :

   CREATE TABLE table_name (

   column_1 datatype,

   column_2 datatype,

   ....

   );

   Eg: CREATE TABLE Student_info (

   College_Id number(2),

   College_name varchar(30),

   );

2. **Alter**

   This command is used to add, delete or change columns in the existing table. The user needs to know the existing table name and can do add, delete or modify tasks easily.

   Syntax:

   ALTER TABLE table_name

   ADD column_name datatype;

Eg: ALTER TABLE Student_info

ADD CGPA number;

3. **Truncate:**

This command is used to remove all rows from the table, but the structure of the table still exists.

Syntax:

TRUNCATE TABLE table_name;

Eg: TRUNCATE TABLE Student_info;

4. **Drop:**

This command is used to remove an existing table along with its structure from the database.

Syntax: DROP TABLE table_name;
Eg: DROP TABLE Student_info;

5. **Rename:**

It is possible to change name of table with or without data in it using simple RENAME command.

Syntax: RENAME TABLE <Table Name> To <New_Table_Name>;
Eg: RENAME TABLE Employee To EMP;

VIEW:

A view in SQL is a saved SQL query that acts as a virtual table. It can fetch data from one or more tables and present it in a customized format, allowing developers to Simplify Complex Queries, Enhance Security, Present Data Flexibly.

Syntax:

CREATE VIEW view_name AS
SELECT column1, column2…..
FROM table_name
WHERE condition;

Eg:

```
CREATE VIEW StudentNames AS
SELECT S_ID, NAME
FROM StudentDetails
ORDER BY NAME;
```

Output:

S_ID	NAMES
2	Ashish
4	Dhanraj
1	Harsh
3	Pratik
5	Ram

INDEX:

An index in SQL is a schema object that improves the speed of data retrieval operations on a table. It works by creating a separate data structure that provides pointers to the rows in a table, making it faster to look up rows based on specific column values.It is a critical feature **for** optimizing query performance, especially for large datasets.

Syntax:

CREATE INDEX index
ON TABLE column;

Eg: *CREATE INDEX idx_product_id*
ON Sales (product_id);

CONSTRAINTS:

Constraints are the rules that we can apply on the type of data in a table. That is, we can specify the limit on the type of data that can be stored in a particular column in a table using constraints.

Syntax:

```
CREATE TABLE sample_table
(
column1 data_type(size) constraint_name,
column2 data_type(size) constraint_name,
....
);
```

1. **NOT NULL**: This constraint tells that we cannot store a null value in a column. That is, if a column is specified as NOT NULL then we will not be able to store null in this particular column any more.

```
CREATE TABLE Student
(
ID int(6) NOT NULL,
NAME varchar(10) NOT NULL,
ADDRESS varchar(20)
);
```

2. **UNIQUE**: This constraint when specified with a column, tells that all the values in the column must be unique. That is, the values in any row of a column must not be repeated.

```
CREATE TABLE Student
(
ID int(6) NOT NULL UNIQUE,
```

```
NAME varchar(10),
ADDRESS varchar(20)
);
```

3. **PRIMARY KEY**: A primary key is a field which can uniquely identify each row in a table. And this constraint is used to specify a field in a table as primary key.

```
CREATE TABLE Student
(
ID int(6) NOT NULL UNIQUE,
NAME varchar(10),
ADDRESS varchar(20),
PRIMARY KEY(ID)
);
```

4. **FOREIGN KEY**: A Foreign key is a field which can uniquely identify each row in a another table. And this constraint is used to specify a field as Foreign key.

```
CREATE TABLE Orders
(
O_ID int NOT NULL,
ORDER_NO int NOT NULL,
C_ID int,
PRIMARY KEY (O_ID),
FOREIGN KEY (C_ID) REFERENCES Customers(C_ID)
)
```

Conclusion:

The SQL DDL statements demonstrate how to design and manage a relational database. The tables store and organize data with enforced relationships through primary and foreign keys, while views simplify complex queries. Indexes optimize query performance, sequences automate unique value generation, and synonyms provide flexible aliases for tables. Constraints ensure data integrity by enforcing rules like uniqueness and non-null values.

EXPERIMENT NO : 03

Aim : SQL Queries: Write at least 10 SQL queries on the suitable database application using SQL DML statements.

Outcomes :

1. Clear Database structure
2. Improves communication
3. Apply Constraints for Data Integrity
4. Design and Create Tables
5. Implement Views for Data Abstraction

Software Requirements : MySQL Workbench

Theory :

The SQL commands that deal with the manipulation of data present in the database belong to DML (Data Manipulation Language) and this includes most of the SQL statements. The DML commands in SQL change the data present in the SQL database. We can easily access, store, modify, update and delete the existing records from the database using DML commands.

1) INSERT:

INSERT is another most important data manipulation command in Structured Query Language, which allows users to insert data in database tables.

Syntax:

INSERT INTO TABLE_NAME (column_Name1 , column_Name2 ,... column_NameN) VALUES (value_1, value_2,... value_N) ;

Eg:
INSERT INTO Student (Stu_id, Stu_Name, Stu_Marks, Stu_Age) VALUES (104, Anmol, 89, 19);

2) UPDATE:

UPDATE is another most important data manipulation command in SQL, which allows users to update or modify the existing data in database tables.

Syntax:
UPDATE Table_name SET[column_name1= value_1, ….., column_nameN = value_N]Where C ondition;

Eg: UPDATE Product SET Product_Price = 80 WHERE Product_Id = 'P102' ;

3) DELETE:

DELETE is a DML command which allows SQL users to remove single or multiple existing records from the database tables. It does not delete the stored data permanently from the database. We use the WHERE clause with the DELETE command to select specific rows from the table.

Syntax: DELETE FROM Table_Name WHERE condition;

Eg: DELETE FROM Product WHERE Product_Id = 'P202' ;

4) SELECT:

SELECT is the most important data manipulation command in Structured Query Language. The SELECT command shows the records of the specified table. It also shows the particular record of a particular column by using the WHERE clause.

Syntax:
SELECT column_Name_1, column_Name_2, ….., column_Name_N FROM Name_of_table;

Eg: SELECT * FROM Student;

OPERATORS:

1) And:

The SQL 'AND' is used to filter records that meet all specified conditions in a Where clause. This logical operator returns a record only when all conditions connected by AND are TRUE. If any of the conditions are FALSE, the record will not be included in the result set. The AND operator allows you to filter data based on multiple conditions, all of which must be true for the record to be included in the result set.

Syntax: *SELECT column1, column2, ...*
FROM table_name
WHERE condition1 AND condition2 AND condition3 ...;

Eg:

```
SELECT * FROM Student
WHERE Age = 18 AND ADDRESS = 'Delhi';
```

2) Or:

The OR Operator in SQL displays the records where any one condition is true, i.e. either condition1 or condition2 is True.

Syntax:

*SELECT * FROM table_name WHERE condition1 OR condition2 OR… conditionN;*

Eg: *SELECT * FROM Student WHERE NAME = 'Ram' OR NAME = 'SUJIT';*

3) Between:

The SQL BETWEEN operator is used to test whether a value falls within a given range of values (inclusive). The values can be text, date, or numbers. It can be used in a Select, INSERT, Update or Delete statement. The SQL BETWEEN Condition will return the records where the expression is within the range of value1 and value2.

Syntax:

SELECT column_name(s)
FROM table_name
WHERE column_name BETWEEN value1 AND value2;

Eg: *SELECT Name FROM Emp WHERE Salary BETWEEN 30000 AND 45000;*

4) In:

IN operator allows us to easily test if the expression matches any value in the list of values. It is used to remove the need for multiple OR conditions in Select, Insert, Update, or Delete. We can also use NOT IN to exclude the rows in our list. We should note that any kind of duplicate entry will be retained.

Syntax:

SELECT column_name(s)
FROM table_name
WHERE column_name IN (list_of_values);

Eg:

SELECT Name FROM Emp
WHERE Salary IN (30000, 40000, 25000);

5) Like:

SQL LIKE operator is used with the WHERE clause to search for a specified pattern in a column. LIKE operator finds and returns the rows that fit in the given pattern.

Syntax:

SELECT column1, column2, …
FROM table_name
WHERE column_name LIKE pattern;

Eg:

SELECT * FROM products WHERE name LIKE BINARY 'a%'

Conclusion:

These SQL DML queries illustrate essential operations in managing data within a database, focusing on common actions such as querying data, inserting new records, updating existing records, and deleting records. They are useful for a variety of database applications, such as an online store's management system, where maintaining customer and order data is crucial. Mastery of these SQL queries helps in efficient data manipulation and is foundational to any data-driven application.

EXPERIMENT NO : 04

Aim : SQL Queries: All types of Join, Sub-Query and View: Write at least10 SQL queries for suitable database application using SQL DML statements.

Outcomes :

1. Clear Database structure

2. Improves communication

3. Efficient Data Retrieval

4. Enhanced Problem Solving

5. Optimized Database Operations

Software Requirements : MySQL Workbench

Theory :

- **JOIN:**
  In SQL, joins are used to combine data from two or more tables based on a related column. They allow querying across multiple tables to retrieve meaningful information. A join matches rows in one table with rows in another using a common column, such as a primary and foreign key. There are several types of joins, including Inner Join, Left Join, Right Join, and Full Outer Join, each serving a specific purpose. Joins are essential for working with relational databases and handling complex queries.

Types of Joins:

1. Inner Join(Simple Join)

- Retrieves rows that have matching values in both tables.
- Excludes rows where there is no match.

Syntax:

SELECT columns FROM Table1 INNER JOIN Table2

ON Table1.column = Table2.column;

Example:

SELECT Customers.Name, Orders.OrderID FROM Customers

INNER JOIN Orders

ON Customers.CustomerID = Orders.CustomerID;

2. Left Join (Left Outer Join)

- Retrieves all rows from the left table and the matching rows from the right table.
- If no match exists, the result will contain NULL for the right table's columns.

Syntax:

SELECT columns FROM Table1 LEFT JOIN Table2

ON Table1.column = Table2.column;

Example:

SELECT Customers.Name, Orders.OrderID FROM Customers LEFT JOIN Orders

ON Customers.CustomerID = Orders.CustomerID;

3. Right Join (Right Outer Join)

- Retrieves all rows from the right table and the matching rows from the left table.
- If no match exists, the result will contain NULL for the left table's columns.

Syntax:

SELECT columns FROM Table1

RIGHT JOIN Table2 ON Table1.column = Table2.column;

Example:

SELECT Customers.Name, Orders.OrderID FROM Customers RIGHT JOIN Orders

ON Customers.CustomerID = Orders.CustomerID;

4. Full Join (Full Outer Join)

- Retrieves all rows where there is a match in either table.
- If there’s no match, columns from one table will show NULL.

Syntax:

SELECT columns FROM Table1 FULL OUTER JOIN Table2

ON Table1.column = Table2.column;

Example:

SELECT Customers.Name, Orders.OrderID FROM Customers FULL OUTER JOIN Orders

ON Customers.CustomerID = Orders.CustomerID;

5. Cross Join

- Produces the Cartesian product of both tables.
- Each row in the first table is combined with every row in the second table.

Syntax:

SELECT columns FROM Table1 CROSS JOIN Table2;

Example:

SELECT Customers.Name, Products.Name AS ProductName FROM Customers

CROSS JOIN Products;

- **Subqueries:**
  A subquery is a query nested inside another SQL query, used to perform intermediate calculations or data fetching. It is often **used in** the **WHERE, HAVING, or FROM** clauses of a statement. Subqueries are commonly **used with SELECT, UPDATE,INSERT and DELETE** statements to achieve complex filtering and data manipulation.

  Syntax:
  SELECT column_name FROM table_name
  WHERE column_name expression operator
  (SELECT column_name FROM table_name WHERE ...);

Subquery with SELECT statement:

In SQL, inner queries or nested queries are used most frequently with the SELECT statement.

Syntax:

SELECT Column_Name1, Column_Name2, ..., Column_NameN
FROM Table_Name WHERE Column_Name Comparison_Operator
(SELECT Column_Name1, Column_Name2, ..., Column_NameN
FROM Table_Name WHERE condition;

Example:

SELECT * FROM Student_Details WHERE Stu_Marks**>** (SELECT AVG(Stu_Marks) FRO M Student_Details);

Subquery with the INSERT statement

We can also use the subqueries and nested queries with the INSERT statement in SQL. We can insert the results of the subquery into the table of the outer query.

Syntax:

INSERT INTO Table_Name SELECT * FROM Table_Name WHERE Column_Name Operator (Su bquery);

Example:

INSERT INTO New_Employee SELECT * FROM Old_Employee WHERE Emp_Salary > 40000;

- **Subquery with the UPDATE statement**

The subqueries and nested queries can be used with the UPDATE statement in Structured Query Language for updating the columns of the existing table. We can easily update one or more columns using a subquery with the UPDATE statement.

Syntax:

UPDATE Table_Name SET Column_Name = New_value WHERE Value OPERATOR (SELECT COLUMN_NAME FROM TABLE_NAME WHERE Condition) ;

Example:

UPDATE Employee_Details SET Emp_SalaryEmp_Salary = Emp_Salary + 5000 WHERE Emp_ID IN (SELECT Emp_ID FROM Department WHERE Dept_Grade = 'A') ;

- **Subquery with the DELETE statement**

We can easily delete one or more records from the SQL table using Subquery with the DELETE statement in Structured Query Language.

Syntax :

DELETE FROM Table_Name WHERE Value OPERATOR (SELECT COLUMN_NAME FROM TABLE _NAME WHERE Condition) ;

Example: DELETE FROM Employee_Details WHERE Emp_ID IN (SELECT Emp_ID FROM Depa rtment WHERE Dept_Grade = 'C') ;

- **VIEW:**

A view in SQL is a saved SQL query that acts as a virtual table. It can fetch data from one or more tables and present it in a customized format, allowing developers to Simplify Complex Queries, Enhance Security, Present Data Flexibly.

Syntax:

CREATE VIEW view_name AS

SELECT column1, column2….. FROM table_name WHERE condition;

Example: CREATE VIEW StudentNames AS SELECT S_ID, NAME FROM StudentDetails ORDER BY NAME;

Output:

S_ID	NAME	ADDRESS
1	Harsh	Kolkata
2	Ashish	Durgapur
3	Pratik	Delhi
4	Dhanraj	Bihar
5	Ram	Rajasthan

EXPERIMENT NO : 05

Aim : PL/SQL code block: Use of control structure & Exception handling.

Outcomes :

1. Improved Database Performance
2. Improves communication
3. Enhanced SQL Functionality
4. Better Exception Handling
5. Increased Code Reusability

Software Requirements: MySQL Workbench

Theory:

PL/SQL:

PL/SQL (Procedural Language/Structured Query Language) is Oracle's procedural extension for SQL. It allows users to write code that includes procedural constructs like loops, conditions, and exception handling, making SQL more powerful for data processing.

- **It is mainly used for:**
  - Writing stored procedures, functions, and triggers
  - Implementing business logic within the database
  - Enhancing performance through procedural execution
  - Ensuring better security and code reusability

- **Features of PL/SQL:**

1. **Tight Integration with SQL**
   - PL/SQL supports all SQL commands, including SELECT, INSERT, UPDATE, & DELETE.
   - It allows embedding SQL queries directly into PL/SQL blocks.
2. **Block Structure**
   - PL/SQL code is divided into blocks, improving readability and maintainability.
   - Each block consists of a **DECLARE**, **BEGIN**, **EXCEPTION**, and **END** section.
3. **Procedural Capabilities**
   - Supports procedural constructs like loops (FOR, WHILE), conditional statements (IF-THEN-ELSE), and case statements.
4. **Exception Handling**
   - Provides robust error handling using the EXCEPTION block.
   - Predefined and user-defined exceptions help manage errors efficiently.
5. **Code Reusability**
   - PL/SQL supports stored procedures, functions, and packages, promoting modular and reusable code.

6. **Triggers and Cursors**
    - **Triggers** allow automatic execution of code in response to database events.
    - **Cursors** enable row-by-row processing of query results.
7. **Portability**
    - PL/SQL is platform-independent as it runs within the Oracle database environment.
8. **Object-Oriented Features**
    - Supports object types, enabling object-oriented programming concepts like encapsulation and polymorphism.

- **Syntax:**

```
DECLARE
    declaration statements;

BEGIN
    executable statements

EXCEPTIONS
    exception handling statements

END;
```

- **Example:**

Hello PL/SQL Program

```
DECLARE
  message VARCHAR2(50);                -- Declare a variable
BEGIN
  message := 'Hello, PL/SQL!';         -- Assign a value
  DBMS_OUTPUT.PUT_LINE(message);       -- Print the message
END;
/
```

- **Explanation:**

- DECLARE: Declares variables.
- BEGIN: Marks the start of the execution block.
- DBMS_OUTPUT.PUT_LINE: Used to display output.
- END;: Marks the end of the block.
- / (slash): Executes the PL/SQL block.

Control Structure:

Control structures help control the flow of execution in a PL/SQL program. These are categorized into:

1. **Conditional Control (IF-THEN-ELSE)**
2. **Iterative Control (Loops: FOR, WHILE, LOOP)**
3. **Sequential Control (GOTO, CONTINUE, EXIT)**

Control Structure	Description	Example
IF-THEN-ELSE	Executes statements based on a condition	`IF salary > 50000 THEN ...`
LOOP...EXIT	Executes a block repeatedly until a condition is met	`EXIT WHEN i > 5;`
WHILE Loop	Runs while a condition is `TRUE`	`WHILE i < 10 LOOP ...`
FOR Loop	Iterates a fixed number of times	`FOR i IN 1..10 LOOP ...`
GOTO	Jumps to a labeled section	`GOTO label_name;`
CONTINUE	Skips the current iteration and proceeds to the next	`CONTINUE WHEN i = 3;`

Exception Handling:

An exception is an error which disrupts the normal flow of program instructions. PL/SQL provides us the exception block which raises the exception thus helping the programmer to find out the fault and resolve it.

There are two types of exceptions defined in PL/SQL

1. **User defined exception.**
2. **System defined exceptions.**

Syntax :

```
WHEN exception THEN
    statement;
```

```
DECLARE
  declarations section;
BEGIN
  executable command(s);
EXCEPTION
   WHEN exception1 THEN
     statement1;
   WHEN exception2 THEN
     statement2;
   [WHEN others THEN]
   /* default exception handling code */
END;
```

1)System defined Exception:

These exceptions are predefined in PL/SQL which get raised WHEN certain **database rule is violated.**

Example:

```
DECLARE
  a int:=10;
  b int:=0;
  answer int;

BEGIN
  answer:=a/b;
  dbms_output.put_line('the result after division is'||answer);

exception
  WHEN zero_divide THEN
    dbms_output.put_line('dividing by zero please check the values again');
    dbms_output.put_line('the value of a is '||a);
    dbms_output.put_line('the value of b is '||b);
END;
```

Output:

Error
Change data type of temp to varchar(20)

2) User defined exception:

This type of users can create their own exceptions according to the need and to raise these exceptions explicitly ***raise*** command is used.

Example:

```
DECLARE
   insufficient_balance EXCEPTION;  -- Declare exception
   balance NUMBER := 500;
BEGIN
   IF balance < 1000 THEN
      RAISE insufficient_balance;  -- Raise exception
   END IF;

   DBMS_OUTPUT.PUT_LINE('Transaction successful.');
EXCEPTION
   WHEN insufficient_balance THEN
      DBMS_OUTPUT.PUT_LINE('Error: Insufficient balance.');
END;
/
```

Output:

Error: Insufficient balance.

EXPERIMENT NO : 06

Aim : PL/SQL code block: Parameterized Cursor .

Outcomes :

1. Improved Database Performance
2. Flexible Execution
3. Enhanced SQL Functionality
4. Reduces Hardcoding
5. Increased Code Reusability

Software Requirements: MySQL Workbench

Theory:

PL/SQL:

PL/SQL (Procedural Language/Structured Query Language) is Oracle's procedural extension for SQL. It allows users to write code that includes procedural constructs like loops, conditions, and exception handling, making SQL more powerful for data processing.

- **Features of PL/SQL:**

1. Tight Integration with SQL
2. Block Structure
3. Procedural Capabilities
4. Exception Handling
5. Code Reusability
6. Triggers and Cursors
7. Portability
8. Object-Oriented Features

- **Syntax:**

```
DECLARE
   declaration statements;
 BEGIN
   executable statements

EXCEPTIONS
   exception handling statements

END;
```

- **Example:**

Hello PL/SQL Program DECLARE

```
   message VARCHAR2(50);                  -- Declare a variable
BEGIN
   message := 'Hello, PL/SQL!';           -- Assign a value
   DBMS_OUTPUT.PUT_LINE(message);         -- Print the message
END;
```

Control Structure:

Control structures help control the flow of execution in a PL/SQL program.

1. **Conditional Control (IF-THEN-ELSE)**

(a) IF-THEN Statement:

Used inside a stored procedure to check a single condition.

```
DELIMITER $$

CREATE PROCEDURE CheckSalary()
BEGIN
  DECLARE salary INT DEFAULT 5000;

  IF salary > 4000 THEN
    SELECT 'Salary is above 4000' AS Message;
  END IF;
END$$
DELIMITER ;
CALL CheckSalary();
```

(b) IF-THEN-ELSE Statement:

Used when we have two possible outcomes.

```
DELIMITER $$

CREATE PROCEDURE CheckSalaryCondition()

BEGIN
  DECLARE salary INT DEFAULT 3500;

  IF salary > 4000 THEN
    SELECT 'Salary is above 4000' AS Message;
  ELSE
    SELECT 'Salary is 4000 or below' AS Message;
  END IF;
END$$
```

```
DELIMITER ;
CALL CheckSalaryCondition();
```

(c) IF-THEN-ELSIF-ELSE Statement

Used when there are multiple conditions to check.

```
DELIMITER $$

CREATE PROCEDURE CheckGrade()
BEGIN
   DECLARE marks INT DEFAULT 85;
   DECLARE grade VARCHAR(10);

   IF marks >= 90 THEN
      SET grade = 'A';
   ELSIF marks >= 75 THEN
      SET grade = 'B';
   ELSE
      SET grade = 'C';
   END IF;

   SELECT CONCAT('Grade: ', grade) AS Message;
END$$
DELIMITER ;
CALL CheckGrade();
```

Parameterized Cursor:

A Parameterized Cursor in PL/SQL allows passing parameters to a cursor at runtime, making it more flexible and reusable.

Syntax :

```
 DECLARE cursor_name CURSOR FOR
SELECT column1, column2 FROM table_name WHERE condition;
```

Example:

```
DELIMITER $$

CREATE PROCEDURE GetEmployeesByDept(IN dept_no INT)
BEGIN
   DECLARE v_empno INT;
   DECLARE v_ename VARCHAR(50);
   DECLARE v_job VARCHAR(50);
   DECLARE v_sal DECIMAL(10,2);
```

```
    DECLARE emp_cursor CURSOR FOR
     SELECT empno, ename, job, sal FROM emp WHERE deptno = dept_no;

   DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_empno = NULL;
  OPEN emp_cursor;
  read_loop: LOOP
  FETCH emp_cursor INTO v_empno, v_ename, v_job, v_sal;

     IF v_empno IS NULL THEN
        LEAVE read_loop;
     END IF;
          SELECT v_empno AS 'Emp No', v_ename AS 'Name', v_job AS 'Job', v_sal AS 'Salary';
  END LOOP;

  CLOSE emp_cursor;
END$$
DELIMITER ;
CALL GetEmployeesByDept(10);
```

Output:

Expected Output (Assuming Data in EMP Table for DEPTNO = 10):

Emp No	Name	Job	Salary
7369	SMITH	CLERK	800
7782	CLARK	MANAGER	2450
7839	KING	PRESIDENT	5000

Conclusion:

I have learned that parameterized cursors in PL/SQL allow passing values at runtime, making queries more flexible and reusable. They help in retrieving specific data efficiently without hardcoding values, improving database operations.

EXPERIMENT NO : 07

Aim : MongoDB Queries: CRUD operations .

Outcomes :

1. Improved Database Performance
2. Flexible Execution
3. Enhanced SQL Functionality
4. Understanding NoSQL & MongoDB Basics
5. Querying Data Efficiently

Software Requirements: MySQL Workbench

Theory:

NoSQL:

NoSQL, or **"Not Only SQL,"** is a database management system (DBMS) designed to handle large volumes of unstructured and semi-structured data. Unlike traditional relational databases that use tables and pre-defined schemas, NoSQL databases provide **flexible data models** and support **horizontal scalability**, making them ideal for modern applications that require real-time data processing.

- **Why Use NoSQL?**

Traditional relational databases follow ACID (Atomicity, Consistency, Isolation, Durability) principles, ensuring strong consistency and structured relationships between data. However, as applications evolved to handle big data, real-time analytics, and distributed environments, NoSQL emerged as a solution with:

- Scalability
- Flexibility
- High Performance
- Distributed Architecture

MongoDB:

MongoDB the most popular NoSQL database, is an open-source document oriented database. The term 'NoSQL' means 'non-relational'. This means that MongoDB isn't based on a table-like relational database structure but provides an altogether different mechanism for the storage and retrieval of data. This format of storage is called BSON (similar to JSON format).

- Features:
  - Document Oriented
  - Indexing
  - Scalability
  - Replication and High Availability
  - Aggregation

- **Where do we use MongoDB?**

  MongoDB is preferred over RDBMS in the following scenarios:

  - **Big Data**: If we have huge amount of data to be stored in tables, think of MongoDB before RDBMS databases. MongoDB has built-in solution for partitioning and sharding our database.
  - **Unstable Schema**: **Adding a new column** in RDBMS is hard whereas MongoDB is **schema-less**. Adding a new field does not effect old documents and will be very easy.
  - **Distributed data** Since multiple copies of data are stored across different servers, recovery of data is instant and safe even if there is a hardware failure.

CRUD Operations:

CRUD operations Create, Read, Update, and Delete—are essential for interacting with databases. In MongoDB, CRUD operations allow users to perform various actions like inserting new documents, reading data, updating records, and deleting documents from collections.

What are CRUD Operations in MongoDB?

- Create: Add new documents to a collection.
- Read: Retrieve documents from a collection.
- Update: Modify existing documents.
- Delete: Remove documents from a collection.

1)Create:

MongoDB Query (Insert Document):

```
db.users.insertOne({
 name: "John Doe",
 age: 30,
 email: "john@example.com" });
```

MongoDB Output

```
{
  "acknowledged": true,
  "insertedId": ObjectId("65e10fbcad07f77bfae6b8a1")
}
```

Equivalent SQL Query in MySQL Workbench

```
INSERT INTO users (name, age, email) VALUES ('John Doe', 30, 'john@example.com');
```

2) Read:

MongoDB Query (Find Documents)

```
db.users.find();
```

MongoDB Output

```
[
  { "_id": ObjectId("65e10fbcad07f77bfae6b8a1"), "name": "John Doe", "age": 30, "email": "john@example.com" }
]
```

Equivalent SQL Query in MySQL Workbench

```
SELECT * FROM users;
```

3) Update:

MongoDB Query (Update a Document)

```
db.users.updateOne(
  { name: "John Doe" },
  { $set: { age: 31 } }
);
```

MongoDB Output

```
{
  "acknowledged": true,
  "matchedCount": 1,
  "modifiedCount": 1
}
```

Equivalent SQL Query in MySQL Workbench

```
UPDATE users SET age = 31 WHERE name = 'John Doe';
```

4) Delete:

MongoDB Query (Delete a Document)

```
db.users.deleteOne({ name: "John Doe" });
```

MongoDB Output

```
{
  "acknowledged": true,
  "deletedCount": 1
}
```

Equivalent SQL Query in MySQL Workbench

```
DELETE FROM users WHERE name = 'John Doe';
```

Logical Operators:

Logical operators in MongoDB are used to combine multiple query conditions. They help filter documents based on multiple criteria using AND, OR, NOT, and NOR conditions.

List of Logical Operators

Operator	Description
`$and`	Returns documents that match all conditions.
`$or`	Returns documents that match at least one condition.
`$not`	Inverts the effect of a query expression.
`$nor`	Returns documents that do **not** match any condition.

Operator	Usage	Example Query in MongoDB
`$and`	Matches all conditions	`{ $and: [ { age: { $gt: 25 } }, { city: "NY" } ] }`
`$or`	Matches at least one condition	`{ $or: [ { age: { $lt: 20 } }, { city: "LA" } ] }`
`$not`	Inverts a condition	`{ age: { $not: { $gt: 30 } } }`
`$nor`	Matches none of the conditions	`{ $nor: [ { age: { $lt: 20 } }, { city: "LA" } ] }`

Conclusion:

Practicing MongoDB CRUD operations helps in efficiently managing data. It enhances skills in data handling, optimization, and error management, essential for building scalable applications.

EXPERIMENT NO : 08

Aim : MongoDB Queries: Aggregation and Indexing .

Outcomes :

1. Improved Database Performance
2. Flexible Execution
3. Enhanced SQL Functionality
4. Real-Time Analytics
5. Querying Data Efficiently

Software Requirements: MySQL Workbench

Theory:

Aggregation:

MongoDB Aggregation is a database process that allows us to perform complex data transformations and computations on collections of documents or rows. It enables us to group, filter & manipulate data to produce summarized results. It is typically carried out using the aggregation pipeline which is a framework for data aggregation modeled on the concept of data processing pipelines.

Each stage of the pipeline transforms the documents as they pass through it and allows for operations like filtering, grouping, sorting, reshaping and performing calculations on the data.

- **Example:**

Counting Users in each city

Query:

```
db.users.aggregate([
  { $group: { _id: "$city", totalUsers: { $sum: 1 } } }
])
```

Output:

```
[
  { _id: 'Los Angeles', totalUsers: 1 },
  { _id: 'New York', totalUsers: 1 },
  { _id: 'Chicago', totalUsers: 1 }
]
```

- **Advantages of Using MongoDB Aggregation:**
  - Reduces Network Load
  - Complex Data Transformation
  - Index Optimization
  - Supports Real-Time Analytics
  - Scalable for Big Data
  - Flexibility with Stages
  - Eliminates Redundant Queries

Indexing:

Indexing in MongoDB is a crucial feature that enhances query processing efficiency. Without indexing, MongoDB must scan every document in a collection to retrieve the matching documents and leading to slower query performance. Indexes are special data structures that store information about the documents in a way that makes it easier for MongoDB to quickly locate the right data.

- **Creating an Index:**
  MongoDB provides a method called createIndex() that allows users to create an index.
  Syntax:
  db.COLLECTION_NAME.createIndex({KEY:1})

- **Drop an index :**
  In order to drop an index, MongoDB provides the dropIndex() method.
  Syntax:
  db.NAME_OF_COLLECTION.dropIndex({KEY:1})

- **Get Description of All Indexes:**
  The getIndexes() method in MongoDB gives a description of all the indexes that exists in the given collection.
  Syntax:
  db.NAME_OF_COLLECTION.getIndexes()

  Example:
  Scenario: Library Management System
  We have two collections:
  1. **books** → Stores book details.
  2. **borrowed_books** → Tracks borrowed books.

  Step 1: Sample Data:
  books Collection:

```
[
  { "_id": 1, "title": "MongoDB Guide", "author": "John Doe", "genre": "Technology",
"price": 30, "stock": 10 },
```

{ "_id": 2, "title": "Python Basics", "author": "Alice Smith", "genre": "Programming", "price": 25, "stock": 15 },
{ "_id": 3, "title": "Fictional World", "author": "David Johnson", "genre": "Fiction", "price": 20, "stock": 5 }
]

borrowed_books Collection:
[
{ "_id": 101, "bookId": 1, "borrower": "Alice", "borrowDate": "2024-03-01", "returnDate": "2024-03-15" },
{ "_id": 102, "bookId": 2, "borrower": "Bob", "borrowDate": "2024-03-05", "returnDate": "2024-03-20" },
{ "_id": 103, "bookId": 1, "borrower": "Charlie", "borrowDate": "2024-03-10", "returnDate": "2024-03-25" }

Step 2: Create Indexes:

Index on genre in books (for fast searching):
```
db.books.createIndex({ genre: 1 });
```

Index on borrowDate in borrowed_books (for fast retrieval of recent borrows)
```
db.borrowed_books.createIndex({ borrowDate: -1 });
```

Step 3: Aggregation Queries:
1. Count of Books Borrowed Per Book:
```
db.borrowed_books.aggregate([
 {
  $group: {
   _id: "$bookId",
   totalBorrows: { $sum: 1 }
  }
 }
]);
```

2. Most Borrowed Books (Top 1 Book):
```
db.borrowed_books.aggregate([
 {
  $group: {
   _id: "$bookId",
   totalBorrows: { $sum: 1 }
  }
 },
 { $sort: { totalBorrows: -1 } },
 { $limit: 1 }
]);
```

3. Join Borrowed Books with Book Details ($lookup):

```
db.borrowed_books.aggregate([
 {
  $lookup: {
   from: "books",
   localField: "bookId",
   foreignField: "_id",
   as: "bookDetails"
  }
 },
 { $unwind: "$bookDetails" }
]);
```

4. Monthly Borrowing Report:

```
db.borrowed_books.aggregate([
 {
  $group: {
   _id: { $substr: ["$borrowDate", 0, 7] },
   totalBorrows: { $sum: 1 }
  }
 },
 { $sort: { _id: 1 } }
]);
```

Conclusion:

MongoDB **Indexing** boosts query performance, while **Aggregation** enables efficient data processing. Together, they optimize data retrieval, analysis, and scalability, making MongoDB ideal for real-world applications like e-commerce and analytics.

EXPERIMENT NO : 09

Aim : MongoDB Queries: Aggregation and Indexing .

Outcomes :

1. Improved Query Performance
2. Optimization of Large Datasets
3. Simplified Complex Queries
4. Real-time Analytics and Reporting
5.Facilitates Data Pipeline Creation

Software Requirements:

A database management system (DBMS) such as MySQL Workbench
SQL client tool (e.g., MySQL Workbench, pgAdmin).
Access to external files such as CSV, TXT, or XLSX files.

Theory:

In a database management system, exporting and importing data are essential tasks for data migration, backup, and data exchange. Exporting involves extracting data from a database and saving it into an external file, whereas importing is the process of loading data from external files back into the database.

1. Exporting Data:

Exporting data can be done in different formats, such as CSV, XLSX, and TXT. SQL provides various methods to extract the data from tables and write it into a file.

CSV (Comma-Separated Values): A text-based format where each row of data is saved as a line in the file, and each column is separated by commas.
XLSX (Excel Spreadsheet): A binary format used by Excel to store tabular data. Tools or specialized queries are used to export data in this format.
TXT (Text File): A plain text format that can hold data in a readable format. Columns may be separated by delimiters like tabs or spaces.

2. Importing Data:

Importing data is the reverse of exporting. It involves reading external files (such as CSV, TXT, or XLSX) and inserting the data into an existing table in a database.

CSV Import: CSV files are commonly imported into relational databases because they are easy to handle and are supported natively by most databases.
TXT Import: Text files may be imported using specific delimiters to define where columns start and end.
XLSX Import: Importing from an Excel file requires specialized functions or tools, as relational databases typically don’t support Excel files natively.

Procedure:

Part 1: Exporting Data
Exporting Data to CSV File:

The following SQL query exports a table (employee) to a CSV file:

```
SELECT * INTO OUTFILE '/path/to/export/employee.csv'
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
FROM employee;
```

Explanation:
INTO OUTFILE: Specifies the output file where the data will be saved.
FIELDS TERMINATED BY ',': Specifies that the columns will be separated by commas.
ENCLOSED BY '"': Specifies that the fields will be enclosed in double quotes.
LINES TERMINATED BY '\n': Specifies that each row will be terminated by a newline.

Exporting Data to TXT File:

The following query exports data to a text file

```
SELECT *  INTO OUTFILE '/path/to/export/employee.txt'
FIELDS TERMINATED BY '\t'  -- Tab-separated
LINES TERMINATED BY '\n'
FROM employee;
```

In this case, columns are separated by tabs ('\t') and rows by newlines ('\n').
Exporting Data to XLSX (Using External Tools):

While SQL itself doesn't support direct export to XLSX, you can use external tools such as mysqldump, pg_dump, or database management tools like MySQL Workbench or pgAdmin, which allow you to export tables directly to Excel format.

Part 2: Importing Data:
Importing Data from CSV File:

The following SQL query imports data from a CSV file into a table (employee):

```
LOAD DATA INFILE '/path/to/import/employee.csv'
INTO TABLE employee
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n';
```

Explanation:
LOAD DATA INFILE: Specifies the file from which data will be loaded.
INTO TABLE: Specifies the target table where the data will be inserted.
FIELDS TERMINATED BY ',': Specifies the delimiter used for columns in the CSV file.

ENCLOSED BY '"': Specifies that the fields are enclosed in double quotes.
LINES TERMINATED BY '\n': Specifies the end of a row in the CSV file.

Importing Data from TXT File:

The following SQL query imports data from a text file:
LOAD DATA INFILE '/path/to/import/employee.txt'
INTO TABLE employee
FIELDS TERMINATED BY '\t'
LINES TERMINATED BY '\n';
In this case, columns are separated by tabs ('\t'), and rows are separated by newlines ('\n').

Importing Data from XLSX:

Importing data from an XLSX file requires either:
Converting the XLSX file to CSV format and using the LOAD DATA INFILE method.
Using external database tools like MySQL Workbench or pgAdmin to directly import XLSX files.

Precautions:
Ensure the file paths are correct and accessible to the database server.
Make sure that the data types in the file match the data types of the target columns in the table.
Verify that the file formats and delimiters are consistent with the table structure to avoid import errors.
Backup the database before importing large datasets to avoid accidental data loss.

Conclusion:
This experiment demonstrates how to use SQL DML statements to efficiently export data from a database into various file formats (CSV, TXT, XLSX) and how to import data from these file formats into the database. Mastering these techniques is essential for data migration, backup, and integration tasks in real-world database management scenarios.

EXPERIMENT NO : 10

Aim : Database Connectivity: Program to implement MySQL/Oracle database connectivity with any front end language to implement Database navigation operations (add, delete, edit)

Outcomes :

1. Establish Database Connectivity
2. Perform CRUD Operations
3. Enhanced Security
4. Optimize Performance
5. Ensure Data Integrity and Validation

Software Requirements: MySQL Workbench

Theory:

Database Connectivity:

The MySQL database connection management refers to the process of establishing or maintaining and efficiently utilizing connections between the application and MySQL database server. It involves tasks such as establishing new connections reusing existing connections pooling connections for efficient resource utilization managing connection timeouts or handling connection errors and closing connections when they are no longer needed.

Note: If you're using **MySQL Workbench** and want to connect it with a frontend, you can still use Python with **MySQL Connector**.

- **Steps to Implement MySQL Workbench Database Connectivity in Python:**

Step 1: Create a Database in MySQL Workbench

1. Open MySQL Workbench.
2. Connect to your MySQL server.
3. Create a new database:
   CREATE DATABASE mydatabase;
4. Use the database:
   USE mydatabase;

5. Create a table:
   CREATE TABLE users (

```
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255),
    email VARCHAR(255)
);
```

Step 2: Install MySQL Connector in Python:
Ensure you have mysql-connector-python installed:
pip install mysql-connector-python

Step 3: Python Program to Connect to MySQL Workbench:
This program will allow you to **add, view, update, and delete** user records.

```
import mysql.connector
# Establishing connection to MySQL Workbench database
conn = mysql.connector.connect(
    host="localhost",
    user="root",  # Replace with your MySQL username
    password="yourpassword",  # Replace with your MySQL password
    database="mydatabase"
)
cursor = conn.cursor()

# Function to add a user
def add_user(name, email):
    cursor.execute("INSERT INTO users (name, email) VALUES (%s, %s)", (name, email))
    conn.commit()
    print("User added successfully.")

# Function to retrieve all users
def get_users():
    cursor.execute("SELECT * FROM users")
    for row in cursor.fetchall():
        print(row)

# Function to update user details
def update_user(user_id, new_name, new_email):
    cursor.execute("UPDATE users SET name = %s, email = %s WHERE id = %s", (new_name,
new_email, user_id))
    conn.commit()
    print("User updated successfully.")
```

```
# Function to delete a user
def delete_user(user_id):
    cursor.execute("DELETE FROM users WHERE id = %s", (user_id,))
    conn.commit()
    print("User deleted successfully.")

# Command-line interface for CRUD operations
while True:
    print("\nMySQL Workbench Database Navigation")
    print("1. Add User")
    print("2. View Users")
    print("3. Update User")
    print("4. Delete User")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        name = input("Enter name: ")
        email = input("Enter email: ")
        add_user(name, email)

    elif choice == "2":
        get_users()

    elif choice == "3":
        user_id = input("Enter user ID to update: ")
        new_name = input("Enter new name: ")
        new_email = input("Enter new email: ")
        update_user(user_id, new_name, new_email)

    elif choice == "4":
        user_id = input("Enter user ID to delete: ")
        delete_user(user_id)

    elif choice == "5":
        print("Exiting...")
        break

    else:
        print("Invalid choice. Try again.")
```

```
# Close the database connection
cursor.close()
conn.close()
```

Step 4: Running the Program:

1. Save the file as mysql_crud.py.
2. Run it using:
   python mysql_crud.py
3. Use the menu options to **add, view, update, and delete** users.

Step 5: Integrating with a Frontend:

If you want a GUI, you can integrate this with:

- **Tkinter** (Python GUI)
- **Flask/Django** (for Web Apps)
- **React/Angular** (for modern frontends)

Conclusion:

This program enables seamless MySQL/Oracle database connectivity with any frontend, allowing efficient CRUD operations. It ensures data integrity, security, and user-friendly navigation, making it a strong foundation for database-driven applications.